

```
print(i)
if i == 4:
    break
else:
    print("Break from for loop")
```

Figure 2 shows the output of the Python script *loop.py*. Notice that the *else* part of the *for* loop gets executed only when exited normally from the loop and not when exited through a *break* statement. Though not essential, *for-else* is a convenient feature that is absent in most of the programming languages.

```
# python loop.py
0
1
2
3
4
Normal Exit from for loop
0
1
2
3
4
#
```

Figure 2: Output of the Python script *loop.py*

Fork bombing with a Linux shell script

What is the smallest program (in terms of source code size) in any language that comes to your mind, which when executed will crash your system? I am sure the winner will be the following shell script.

```
x( ) { x | x & }; x
```

It defines a function called *x*, which calls the function *x* itself recursively and then pipes this result to another recursive call to *x* in the background. The fourth *x* in the script is a call to the function *x* to begin the bombing. Soon your CPU will be allocating all its time to process just these calls to the function *x*. Just 11 non-white space characters in the script and your system is down, so imagine the surprise of those programmers whose favourite

programming language takes hundreds of characters just to print the message 'Hello World' on the screen.

Warning! If you execute this code on a Linux terminal and do not close the terminal immediately, your system will hang. In such a situation, you will have to force restart your system.

Case-insensitive function names in PHP

Imagine copying the directories named 'SONGS', 'Songs' and 'songs' from your Linux machine to your friend's Windows machine. The Windows file system in general is not case-sensitive and you will be forced to rename 2 of these directories before copying them, because all the three directory names (SONGS, Songs and songs) are the same if treated in a case-insensitive manner. Some programming languages behave like this. A classic example is PHP. Consider the PHP script named *case.php* given below:

```
<!DOCTYPE html>
<html>
<body>

<?php
function PRINTMSG( ) {
    echo "I AM PHP <br>";
}

printmsg( );
PrintMsg( );
PRINTMSG( );
pRiNtMsG( );
PrInTmSg( );

?>

</body>

</html>
```

Function names in PHP are case-insensitive and the lines of code *printmsg()*, *PrintMsg()*, *PRINTMSG()*, *pRiNtMsG()*, and *PrInTmSg()* are all calling the function *PRINTMSG()*.

Figure 3 shows the output of the PHP script *case.php*. Do notice that PHP variable names are case-sensitive like most other programming languages.

```
I AM PHP
I AM PHP
I AM PHP
I AM PHP
I AM PHP
```

Figure 3: Output of the PHP script *case.php*

Implicit type conversion in JavaScript

Imagine the case of a Java programmer learning JavaScript. Due to the similarity in names we might think that this will be an easy task. Even though the syntax is somewhat similar, the transition from Java to JavaScript is not that easy. Java is a strongly typed language where type checking is very rigorous, while on the other hand, JavaScript is a weakly typed language with extensive implicit type conversion. This is one feature of JavaScript that should be avoided if possible. To understand the pitfalls in the implicit type conversion of JavaScript, let us go through the JavaScript script named *type.js* given below. What is the output of the script *type.js*?

```
<!DOCTYPE html>
<html>
<body>

<script>
a = '2' + 1
b = '2' - 1
document.write(a + "<br>");
document.write(b + "<br>");
c = '1' + 2 + 3
d = 1 + 2 + '3'
document.write(c + "<br>");
document.write(d);
</script>

</body>
</html>
```